AIN442 Practicum in Natural Language Processing
BBM497 Introduction to Natural Language Processing Lab.
Spring 2026
Instructor: Prof. Dr. İlyas Çiçekli
TA's: Sümeyye Meryem Taşyürek*, İsmail Furkan Atasoy

Programming Assignment 2 – N-gram Language Modeling

Due date: Sunday, 05-04-2026, 11:59 PM.

1 Introduction

In this assignment, you will implement an **N-gram Language Model** from scratch using Python. Language models estimate the probability of word sequences and are fundamental components of many NLP applications such as speech recognition, machine translation, text generation, and predictive typing.

You are required to implement a language model that learns **unigram**, **bigram**, and **trigram** statistics from a training corpus. Your implementation will then use these statistics to compute token probabilities, sentence probabilities, generate text, and evaluate model quality using **perplexity**.

The goal of this assignment is to help you understand:

- how statistical language models are built,
- how local word context affects probability estimation,
- how smoothing helps with unseen words or unseen word sequences,
- and how language models can be evaluated quantitatively.

2 Requirements

You are required to write a Python program that implements an **N-gram Language Model**. Your program must define a class named `ngramLM`.

The constructor of the class should initialize an empty language model. The model should then be trained on a corpus file and should store the necessary unigram, bigram, and trigram statistics needed for all required computations.

3 Instance Variables

Your `ngramLM` class must contain at least the following instance variables:

- `numOfTokens`: The total number of tokens in the training corpus. All tokens that appear in the corpus, including punctuation and special tokens, should be counted.
- `sizeOfVocab`: The size of the vocabulary, i.e., the number of unique tokens that appear in the training corpus.
- `numOfSentences`: The total number of sentences obtained from the training corpus after tokenization and sentence segmentation.

- `sentences`: A list containing all tokenized sentences in the corpus. Each sentence must be represented as a list of tokens.

Each sentence must be padded with special sentence boundary tokens:

- `<s>` : sentence start token
- `</s>` : sentence end token

Thus, the first token of every sentence must be `<s>` and the last token must be `</s>`.
For example, a tokenized sentence should be stored as:

```
['<s>', 'this', 'is', 'an', 'example', '.', '</s>']
```

In addition to the variables listed above, your class must also contain one or more instance variables that store the counts of:

- unigrams
- bigrams
- trigrams

You may design the internal data structures for storing these counts in any reasonable way (for example, using dictionaries). The choice of data structure is left to the implementation, as long as it supports efficient access to the frequency counts required by the methods described in the following sections.

4 Training the Language Model

Implement the following method in your class:

```
trainFromFile(fn)
```

This method should train the language model using the training file whose name is given by `fn`.
The training file is encoded in UTF-8 and may contain both Turkish and English text.
The method must perform the following steps:

1. **Reading the file**

   The training file must be read line by line. Each line should be processed independently.

2. **Tokenization**

   Each line should be tokenized using the following regular expression:

   ```
   r"""(?x)
   (?:[A-ZÇĞIİÖŞÜ]\.)+
   | \d+(?:\.\d*)?(?:\'\w+)?
   | \w+(?:-\w+)*(?:\'\w+)?
   | \.\.\.
   | [][,;.?():_!#^+$%&><|/{()=}\"\'\\\"\‘-]
   """
   ```

3. **Lowercasing**

   All tokens must be converted to lowercase. Before applying lowercase conversion, the following replacements must be performed to correctly handle Turkish characters:

```
I-> ı
İ-> i
```

This step ensures that tokenization does not incorrectly split Turkish words.

4. **Sentence Segmentation**

   The sequence of tokens must be split into sentences. A sentence is assumed to end when one of the following tokens appears:

```
. ? !
```

   Note that the last sentence of a line may not necessarily end with a sentence-final token.

5. **Handling Empty Lines**

   If a line does not produce any tokens after tokenization, that line should be skipped.

6. **Sentence Padding**

   Each sentence must be padded with the special sentence boundary tokens:

```
<s> sentence start token
</s> sentence end token
```

   Thus, every sentence stored in the model must have the following structure:

```
['<s>', token_1, token_2, ..., token_n, '</s>']
```

7. **Updating Model Statistics**

   After constructing the sentences, the following statistics must be updated:

   - unigram counts
   - bigram counts
   - trigram counts
   - total number of tokens
   - vocabulary size

   These statistics will later be used to compute probabilities and generate sentences.

5 Vocabulary and N-gram Extraction

In this section, you will implement methods that return the vocabulary and the N-gram statistics learned from the training corpus.

5.1 Vocabulary

Implement the following method:

```
vocab()
```

This method should return the vocabulary of the corpus.
Each item in the returned list must be a tuple:

```
(word, frequency)
```

where `frequency` is the number of times the word appears in the training corpus.
The returned list must be sorted according to the following rules:

1. First, sort in **descending order of frequency**.
2. If two words have the same frequency, sort them in **ascending alphabetical order**.

5.2 Bigrams

Implement the following method:

```
bigrams()
```

This method should return the list of bigrams extracted from the training corpus.
Each element of the returned list must be a tuple of the following form:

```
((word1, word2), frequency)
```

where `frequency` is the number of occurrences of the bigram $(word_1, word_2)$ in the corpus.
The returned list must be sorted according to the following rules:

1. First, sort in **descending order of frequency**.
2. If two bigrams have the same frequency, sort them in **ascending lexicographical order**.

5.3 Trigrams

Implement the following method:

```
trigrams()
```

This method should return the list of trigrams extracted from the training corpus.
Each element of the returned list must be a tuple of the following form:

```
((word1, word2, word3), frequency)
```

where `frequency` is the number of occurrences of the trigram $(word_1, word_2, word_3)$ in the corpus.
The returned list must be sorted according to the following rules:

1. First, sort in **descending order of frequency**.
2. If two trigrams have the same frequency, sort them in **ascending lexicographical order**.

These methods allow inspection of the learned statistics and will also be useful for debugging and evaluation.

6 Frequency Methods

In order to compute probabilities, the language model must be able to access the frequency counts of unigrams, bigrams, and trigrams.
Implement the following methods:

```
unigramCount(word)
bigramCount((w1,w2))
trigramCount((w1,w2,w3))
```

6.1 Unigram Count

The method `unigramCount(word)` should return the number of times the given word appears in the training corpus.
Example:

```
lm.unigramCount("the") -> 25
```

If the word does not exist in the vocabulary, the method must return:

```
0
```

6.2 Bigram Count

The method `bigramCount((w1,w2))` should return the number of occurrences of the bigram (w_1, w_2) in the training corpus.
Example:

```
lm.bigramCount(("new","york")) -> 12
```

If the bigram does not appear in the corpus, the method must return:

```
0
```

6.3 Trigram Count

The method `trigramCount((w1,w2,w3))` should return the number of occurrences of the trigram (w_1, w_2, w_3) in the training corpus.
Example:

```
lm.trigramCount(("in","the","city")) -> 4
```

If the trigram does not appear in the corpus, the method must return:

```
0
```

These frequency functions must operate efficiently because they will be repeatedly used during probability calculations and sentence generation.

7 Unsmoothed Probabilities

Using the frequency counts obtained from the training corpus, the language model must compute the probabilities of unigrams, bigrams, and trigrams.
Implement the following methods:

```
unigramProb(word)
bigramProb((w1,w2))
trigramProb((w1,w2,w3))
```

These methods should return the **unsmoothed probability values** computed directly from the observed frequencies in the corpus.

7.1 Unigram Probability

The unigram probability of a word w is defined as:

$$P(w) = \frac{\text{freq}(w)}{\text{numOfTokens}}$$

where:

- $\text{freq}(w)$ is the number of occurrences of the word w in the corpus
- numOfTokens is the total number of tokens in the corpus

If the word does not appear in the corpus, the method must return:

```
0
```

7.2 Bigram Probability

The bigram probability represents the probability of observing word w_2 given that the previous word is w_1:

$$P(w_2 \mid w_1) = \frac{\text{freq}(w_1, w_2)}{\text{freq}(w_1)}$$

where:

- $\text{freq}(w_1, w_2)$ is the number of occurrences of the bigram (w_1, w_2)
- $\text{freq}(w_1)$ is the frequency of the word w_1

If either w_1 or w_2 does not appear in the training corpus, the method must return:

```
0
```

7.3 Trigram Probability

The trigram probability represents the probability of observing word w_3 given the two previous words w_1 and w_2:

$$P(w_3 \mid w_1, w_2) = \frac{\text{freq}(w_1, w_2, w_3)}{\text{freq}(w_1, w_2)}$$

where:

- $\text{freq}(w_1, w_2, w_3)$ is the number of occurrences of the trigram (w_1, w_2, w_3)
- $\text{freq}(w_1, w_2)$ is the frequency of the bigram (w_1, w_2)

If the trigram context (w_1, w_2) does not exist in the training corpus, the method must return:

```
0
```

Note that these probabilities are **unsmoothed**. Therefore, any unseen word or word sequence will result in a probability of zero.

8 Smoothed Probabilities

In practice, many words or word sequences may not appear in the training corpus. To handle such cases, the language model must implement **Add-1 smoothing** (Laplace smoothing). Implement the following methods:

```
unigramProb_SmoothingUNK(word)
bigramProb_SmoothingUNK((w1,w2))
trigramProb_SmoothingUNK((w1,w2,w3))
```

These methods should compute probabilities using Add-1 smoothing and must also handle unknown words.

8.1 Smoothed Unigram Probability

The smoothed unigram probability is defined as:

$$P(w) = \frac{\text{freq}(w) + 1}{\text{numOfTokens} + (|\text{V}| + 1)}$$

where $|\text{V}|$ is the vocabulary size. The additional $+1$ in the denominator accounts for unknown tokens.

8.2 Smoothed Bigram Probability

The smoothed bigram probability is defined as:

$$P(w_2 \mid w_1) = \frac{\text{freq}(w_1, w_2) + 1}{\text{freq}(w_1) + (|\text{V}| + 1)}$$

If w_1 is an unknown word, the probability should be:

$$P(w_2 \mid w_1) = \frac{1}{|\text{V}| + 1}$$

8.3 Smoothed Trigram Probability

The smoothed trigram probability is defined as:

$$P(w_3 \mid w_1, w_2) = \frac{\text{freq}(w_1, w_2, w_3) + 1}{\text{freq}(w_1, w_2) + (|\text{V}| + 1)}$$

If the context (w_1, w_2) is unknown, the probability should be:

$$P(w_3 \mid w_1, w_2) = \frac{1}{|\text{V}| + 1}$$

9 Sentence Probability

Implement the following method:

```
sentenceProb(sent, model="bigram")
```

The method should return the probability of the given sentence using the **smoothed N-gram probabilities** defined in the previous section.
The sentence `sent` will be given as a list of tokens.
Example:

```
['<s>', 'this', 'is', 'a', 'sentence', '.', '</s>']
```

9.1 Bigram Model

If the parameter `model="bigram"` is used, the sentence probability should be computed as:

$$P(W) = \prod_{i=2}^{n} P(w_i \mid w_{i-1})$$

where w_i is the i-th token of the sentence.

9.2 Trigram Model

If the parameter `model="trigram"` is used, the sentence probability should be computed as:

$$P(W) = \prod_{i=3}^{n} P(w_i \mid w_{i-2}, w_{i-1})$$

All probabilities must be computed using the **smoothed probability methods** implemented in the previous section.

10 Perplexity Evaluation

Implement the following method:

```
perplexity(testFile, model="bigram")
```

Perplexity is a standard evaluation metric used to measure how well a language model predicts a sequence of words. A lower perplexity value indicates a better language model.
The perplexity of a sequence W is defined as:

$$PP(W) = \exp\left(-\frac{1}{N}\sum_{i} \log P(w_i \mid \text{context})\right)$$

where:

- N is the total number of tokens in the test corpus
- $P(w_i \mid \text{context})$ is the predicted probability of token w_i

The context depends on the selected model:

- Bigram model: $P(w_i \mid w_{i-1})$
- Trigram model: $P(w_i \mid w_{i-2}, w_{i-1})$

The test file must be processed using the same tokenization and sentence preprocessing steps used for the training data.
All probabilities used in perplexity computation must be obtained using the **smoothed probability methods**.

11 Sentence Generation

Implement the following method:

```
generateSentence(sent=["<s>"], maxFollowWords=3, maxWordsInSent=20)
```

This method generates a sentence using the bigram statistics learned from the training corpus. The generation process starts from the last word of the sentence `sent` (which by default is `<s>`). The next word is selected using a probability-based sampling approach from the most frequent words that follow the current word.

- First, identify the top k candidate words that can follow the current word. The value of k is given by `maxFollowWords`.
- These candidates correspond to the bigrams with the highest frequencies.
- The next word must then be sampled according to the relative bigram frequencies.

For example, suppose the following bigram frequencies exist for a word w:

$$\text{freq}(w, f_1) = 4, \quad \text{freq}(w, f_2) = 3, \quad \text{freq}(w, f_3) = 1$$

Then the next word should be selected with the following probabilities:

$$P(f_1) = \frac{4}{8}, \quad P(f_2) = \frac{3}{8}, \quad P(f_3) = \frac{1}{8}$$

Sentence generation stops when either of the following conditions is satisfied:

- the end-of-sentence token `</s>` is generated
- the sentence length reaches `maxWordsInSent`

When the method is called without any arguments, it should generate the most probable sentence starting from `<s>`.

12 Corpora and Files

The following corpus files are provided with the assignment:

- `train_corpus.txt`: the main training corpus that must be used to train the language model
- `visible_testCorpus.txt`: visible test corpus file that can be used for debugging, sanity checking and evaluation which will be used in grading also.
- `ngramLM_template.py`: a template Python file containing the class structure and the method signatures that must be implemented

In addition to the visible test file, submissions will be evaluated using an additional **hidden test corpus**.
Your implementation must therefore work correctly for any valid input file that follows the assignment format and should not rely on corpus-specific assumptions.
All corpus files are encoded in UTF-8 and contain Turkish text.

12.1 Example Test Case

To demonstrate how your implementation should behave, an example test corpus, input test cases and expected outputs are provided:

- `tinyTestCorpus.txt`
- `tinyTest_input.txt`
- `tinyTest_output.txt`

Important: This example is intended only to illustrate how the functions of the language model are expected to work and how outputs are formatted. This is provided only for debugging purposes. In this case, the same dataset is used for both training and testing to allow easy verification of correctness. **This does not reflect the actual evaluation protocol used in grading.** This dataset is not used for grading. Your implementation will be evaluated using visible and hidden corpora.

12.2 Example Checks on the Visible Test Corpus

The file `visible_testCorpus.txt` is provided so that you can test your implementation on unseen data during development. The following examples illustrate the expected behavior of the language model on this corpus.
The visible test corpus is:

```
Dil modeli kelime olasılığı tahmin eder.
Bigram modeli komşu kelimeleri kullanır.
Trigram modeli daha geniş bağlam kullanır.
Perplexity düşük ise model daha iyidir!
```

After tokenization, lowercasing, and sentence padding, this corpus should produce the following sentence structure:

```
['<s>', 'dil', 'modeli', 'kelime', 'olasılığı', 'tahmin', 'eder', '.', '</s>']
['<s>', 'bigram', 'modeli', 'komşu', 'kelimeleri', 'kullanır', '.', '</s>']
['<s>', 'trigram', 'modeli', 'daha', 'geniş', 'bağlam', 'kullanır', '.', '</s>']
['<s>', 'perplexity', 'düşük', 'ise', 'model', 'daha', 'iyidir', '!', '</s>']
```

Expected Basic Statistics

- `numOfSentences` = 4
- `numOfTokens` = 35
- `sizeOfVocab` = 21

Sample Count Checks

- `unigramCount("modeli")` = 3
- `unigramCount("bilinmeyen")` = 0
- `bigramCount(("dil","modeli"))` = 1
- `bigramCount(("yok","kelime"))` = 0
- `trigramCount(("bigram","modeli","komşu"))` = 1
- `trigramCount(("yok","kelime","üçlü"))` = 0

Sample Probability Checks

- `unigramProb("modeli")` $= \frac{3}{35}$
- `unigramProb("bilinmeyen")` $= 0$
- `bigramProb(("dil","modeli"))` $= 1$
- `bigramProb(("kelimeleri","kullanır"))` $= 1$
- `trigramProb(("bigram","modeli","komşu"))` $= 1$
- `trigramProb(("trigram","modeli","daha"))` $= 1$

Sanity Checks

Your implementation should also satisfy the following:

- Unsmoothed probabilities of unseen words or unseen n-grams should be 0.
- Smoothed probabilities should always be non-zero.
- Sentence probabilities computed with smoothing should be greater than 0.
- Perplexity values should be positive.

Note: The above values are only a small subset of possible checks. Your implementation will be evaluated using additional tests and datasets.

Example Usage

```
lm = ngramLM()
lm.trainFromFile("train_corpus.txt")

print(lm.perplexity("visible_testCorpus.txt", model="bigram"))
print(lm.perplexity("visible_testCorpus.txt", model="trigram"))

print(lm.generateSentence())
```

13 Experiments

After implementing the language model, you must run a small set of experiments using your implementation and the provided corpus files.
Using `train_corpus.txt` as the training corpus, your implementation should compute and report the following:

- Vocabulary size
- Top 50 most frequent bigrams
- Top 50 most frequent trigrams

In addition, evaluate your model on the visible test data using the **perplexity** metric.
Compute the perplexity values of the following models on `visible_testCorpus.txt`:

Model	Perplexity
Unigram	
Bigram	
Trigram	

Include these results in your report and briefly comment on them. In particular, discuss how increasing the context size from unigram to trigram affects the perplexity values.

14 Submission

You must submit the following files:

- `<student id>.py`
  Implementation of the language model.
- `report.pdf`
  A short report (maximum one page).

You should prepare a ZIP file named <student id>.zip containing the `<student id>.py` containing all your codes, and `report.pdf`. Submit it to **HADİ**, where you will be assigned a submission.

14.1 Code (85pts) & Report (15pts)

Your code will be primarily evaluated based on output correctness.
Your report should include:

- experiments and the perplexity table for the three models (unigram, bigram, trigram)
- one or two example sentences generated by your model
- a short discussion of the results (3–5 sentences)

The report should be concise and focus only on the experimental results.

14.2 Late Policy

- You must submit your programming assignment **before its due date**.
- You may submit your assignment up to **three days late**, but with a penalty:
  - 1 day late: 10% penalty
  - 2 days late: 20% penalty
  - 3 days late: 30% penalty

DO YOUR PROGRAMMING ASSIGNMENTS YOURSELF!

- Do not share your programming assignments with your friends.
- Do not complete your entire assignment using AI tools, and clearly comment on any part where you have received support from AI tools.
- Your code will be checked using a similarity check, and any submission with a high degree of similarity will be considered cheating. Using AI tools significantly increases the similarity between your solutions. Therefore, the responsibility for maintaining originality lies entirely with you.